DAA PROJECT — EXECUTION BLUEPRINT

Adaptive Route Optimization in Dynamic Environments

Detailed Phase-Wise Implementation & Research Execution Plan

Version 1.0 — Antigravity Execution Blueprint

1. Executive Decision

Purpose: This document converts the master project idea into an executable plan for a four-person DAA group using Antigravity as the primary development environment. It is intentionally research-first: the software, simulator, experiments, results, and final UI are all built around one reproducible experimental question.

What is fixed: The project studies how a routing solution should be updated when the routing problem changes while execution is already in progress. The core comparison is between lightweight incremental repair, repair plus local search, and stronger re-optimization under controlled computational budgets.

What is not fixed yet: The exact stronger search method, final benchmark mix, computational-budget definition, and whether an adaptive selector is justified are deliberately decided through pilot experiments rather than assumed in advance.

Important scope decision: The project is application-neutral. Delivery/courier routing is used only as an intuitive real-world interpretation and literature context. The implemented research problem is a generic dynamic routing optimization framework, so the same method can be interpreted for delivery, ride dispatch, field service, inspection, maintenance, or other changing routing environments.

2. Research Core — What We Are Actually Building

Problem: Given a current routing solution, new routing information arrives during execution. The system must update the solution quickly while keeping route quality high.

Dynamic event: A new vertex/request becomes available after the current solution has already been constructed. The first implementation will avoid traffic APIs and other unnecessary sources of uncertainty.

Objective: Minimize route travel cost while balancing solution quality against the computation required to update the route.

Primary research question: Under controlled computational budgets, when does incremental repair, local-search repair, or stronger re-optimization provide the best quality–response trade-off as problem size and dynamicity change?

Secondary question: Can observed performance patterns support a simple, interpretable strategy selector that chooses an appropriate update method for an unseen dynamic state?

Contribution type: A controlled, reproducible empirical algorithm study and, if justified by evidence, an interpretable adaptive selection layer. It is not a claim to invent a fundamentally new routing problem or metaheuristic.

3. Literature Evidence Used to Shape the Plan

• Huang & Lu, 2020 — Uses online dynamic courier routing, heuristic insertion, adaptive memory and simulated annealing; compares against greedy and reports response time. Their results show that the useful strategy can depend on dynamism level, and their future work explicitly mentions hyper-heuristic insertion. This supports studying controlled strategy choice, but also means a generic 'dynamic insertion + SA' claim is not novel.

• Costa et al., 2024 — Studies dynamic VRP where new demands appear after initial routes. DSNR uses 2-Opt* to allocate new packages and reports gains over QRP Sweep and K-Means Greedy on Loggibud data. This is a close precedent for dynamic local repair, so our project must not claim that 2-Opt* insertion itself is new.

• Liu & Wang, 2026 — Combines traffic-aware ALNS, layered Tabu memory and rollout dispatch and reports response time, quality and robustness. The work demonstrates the research value of multi-metric dynamic evaluation and ablation, while also showing that a four-person undergraduate project should avoid reproducing the full traffic-aware system.

• Kolachalam & Malhotra, 2026 — Compares greedy efficiency heuristics with ALNS, GA and SA under identical conditions and emphasizes the reward-versus-compute frontier. It reinforces the importance of measuring computational cost, not only final solution quality.

• Curated routing corpus — The project spreadsheet and research notes contain a broad 2019–2026 routing literature corpus. The notes identify recurring evaluation gaps: inconsistent dynamism scales, inconsistent compute budgets, limited

cross-instance-family robustness, incomplete response-time reporting, and non-standardized multi-objective evaluation.

The literature supports the direction but also sets boundaries: online insertion + adaptive memory + SA, dynamic 2-Opt* repair, traffic-aware ALNS/Tabu/rollout, and compute-vs-quality comparisons all already exist. The undergraduate contribution therefore centers on controlled budgets, reproducibility, robustness and evidence-based strategy selection.

4. Research Gaps We Will Target

• Gap 1 — Fair computational comparison — Many studies compare methods under different stopping rules or computational settings. We will explicitly control the computational budget and count update/selection overhead.

• Gap 2 — Controlled dynamism curve — Dynamicity is not consistently standardized. We will define reproducible dynamicity levels and study how performance changes as the fraction/intensity of new events increases.

• Gap 3 — Cross-instance robustness — Results are often tied to one benchmark family or one simulated environment. We will use held-out instance characteristics/families for validation where feasible.

• Gap 4 — Quality vs response — A dynamic algorithm cannot be judged by route cost alone. We will report route cost, update time, computational effort, feasibility, and route disruption.

• Gap 5 — Evidence-based adaptation — Instead of inventing an adaptive selector first, we will test whether the data actually reveal conditions under which different strategies win. Only then will an interpretable selector be built.

5. Final Phase Architecture

Phase 0–1 are treated as the completed research foundation. Phase 2 onward is the implementation program. There is intentionally no separate Phase 11; final documentation/reporting is integrated into Phase 10.

Phase 0 — Project Foundation

Status: Completed / baseline locked

Goal: Freeze the project identity, scope, research question, contribution type, and engineering principles.

Tasks:

- Use the master blueprint as the project source of truth.
- Keep the project application-neutral; delivery is context, not the core implementation.
- Freeze the first dynamic model: new vertices/requests arrive after initial routing.
- Freeze primary metrics: route cost, update time, computational effort, disruption, feasibility and scalability.
- Define the adaptive selector as conditional, not guaranteed.

Deliverables:

- Master blueprint in docs/
- One-page problem statement
- Scope and non-scope list
- Research questions and hypotheses

Phase gate: All four members can explain the problem, why static routing is insufficient, what changes dynamically, and what will be compared.

Phase 1 — Literature & Gap Validation

Status: Completed / research corpus established

Goal: Turn the literature corpus into an evidence-backed research protocol.

Tasks:

- Maintain a paper matrix with problem variant, dynamic mechanism, algorithm, baseline, dataset, budget, metrics, limitations and future work.
- Treat the identified gaps as evaluation/methodology gaps, not as claims that dynamic routing is unexplored.
- Select the closest baselines from the literature that can realistically be implemented.
- Record exact reasons why large systems such as traffic-aware ALNS + rollout are outside the undergraduate scope.

Deliverables:

- Literature matrix
- Research-gap statement
- Final research questions
- Initial experimental protocol

Phase gate: No team member describes the project as 'we invented dynamic routing' or 'we invented 2-opt'.

Phase 2 — Algorithmic & Mathematical Specification

Status: Next

Goal: Convert the research question into an exact computational model before coding the main algorithms.

Tasks:

- Define graph G=(V,E) and edge cost c(i,j).
- Define a route and total route cost.
- Define the dynamic state at time t and the event that changes it.
- Define the initial solution and the update operator U(S_t, G_{t+1}).
- Define dynamicity precisely: for the first version, proportion of requests/vertices revealed after the initial solution.
- Define route disruption precisely as the fraction/number of route edges or positions changed after an event.
- Decide whether the core experiment is single-route dynamic TSP or multi-route dynamic VRP. Recommended default: start with single-route dynamic TSP for correctness, then extend to multi-route only after the core engine is stable.
- Define the objective and all constraints before implementation.

Deliverables:

- Formal problem specification
- Notation sheet
- Pseudocode for every baseline
- Complexity-analysis draft
- Small hand-worked examples

Phase gate: A 10-node example can be manually traced from initial route → dynamic event → updated route → metric calculation.

Phase 3 — Core Routing Engine

Status: Next

Goal: Build a correct, modular, testable static routing engine.

Tasks:

- Implement graph/instance model.
- Implement route representation.
- Implement exact route-cost calculation.
- Implement feasibility validation.
- Implement deterministic random-instance generation with seeds.
- Implement baseline construction such as nearest-neighbor / greedy construction.
- Implement a simple exact solver for very small instances only, so that heuristic correctness can be checked against known optimum.
- Add route plotting independent of the final UI.
- Write unit tests before adding dynamic behavior.

Deliverables:

- src/models/
- src/algorithms/baselines/
- src/evaluation/
- tests/
- small-instance correctness suite

Phase gate: All baseline tests pass; cost calculations are independently verified; exact-vs-heuristic comparison works on small instances.

Phase 4 — Local Search & Repair Algorithms

Status: Next

Goal: Implement the algorithmic baselines that make the research comparison meaningful.

Tasks:

- Implement 2-opt as a local-search improvement method.
- Implement dynamic insertion/repair: place a new vertex at the least additional-cost feasible position.
- Implement repair + 2-opt as a second strategy.
- Instrument number of candidate evaluations, accepted moves and improvement iterations.
- Keep algorithms deterministic when given the same seed/configuration.
- Do not add GA/SA/ALNS unless pilot results show that a stronger comparator is necessary and feasible.

Deliverables:

- Greedy/insertion repair
- Repair + 2-opt
- Algorithm-level benchmarks
- Complexity notes
- Unit and integration tests

Phase gate: For the same dynamic event, both repair methods produce valid routes and their cost/time measurements are reproducible.

Phase 5 — Dynamic Simulation Engine

Status: Next

Goal: Create the controlled environment in which the actual research question can be tested.

Tasks:

- Generate a complete hidden instance.
- Reveal an initial subset and build the initial route.
- Generate future dynamic events from a recorded seed.
- Release new vertices at controlled event times/steps.
- Freeze already-traversed portions if the model requires it; only uncommitted route portions may be repaired.
- Run each strategy on exactly the same instance and event sequence.
- Log every event and route state.
- Support replay of any experiment from its seed/configuration.

Deliverables:

- Event-driven simulator
- Dynamic state model
- Replay system
- Event logs

- Animated route-state prototype

Phase gate: A complete run can be replayed exactly and produces the same event sequence and results from the same seed.

Phase 6 — Controlled Experimental Framework

Status: Next

Goal: Turn the simulator into a reproducible research laboratory.

Tasks:

- Define pilot sizes and dynamicity levels.
- Recommended initial dynamicity: 10%, 20%, 40%, 60%, 80%; refine after pilot runs if necessary.
- Recommended pilot sizes: 50, 100, 200 nodes; expand only if runtime permits.
- Choose one primary computational budget: fixed wall-clock time OR fixed evaluation count. Record the reason.
- Run multiple seeds per condition; start with 10 and increase if variance is high and time permits.
- Store raw results in structured CSV/Parquet/JSON files with configuration and seed metadata.
- Never compare algorithms using different instance/event sequences.
- Track implementation version/commit hash.

Deliverables:

- Experiment configuration files
- Batch runner
- Raw result schema
- Experiment manifest
- Reproducibility script

Phase gate: A single command can execute a complete experiment matrix and produce raw results without manual intervention.

Phase 7 — Research Analysis & Ablation

Status: Next

Goal: Determine what the data actually say before designing any adaptive method.

Tasks:

- Compare route quality under equal budgets.
- Compare update response time.
- Compare computational effort.
- Measure route disruption.
- Analyze scaling with problem size.
- Analyze scaling with dynamicity.
- Perform ablations: insertion only vs insertion + 2-opt; adaptation overhead vs no overhead where relevant.
- Use repeated runs and uncertainty summaries rather than one lucky run.
- Use paired comparisons when the same seed/instance is shared across algorithms.
- Create convergence curves and quality-vs-time plots.
- Identify regions where each strategy is clearly better, equivalent, or worse.

Deliverables:

- Clean analysis dataset
- Research plots
- Statistical summaries

- Ablation results
- Findings document

Phase gate: The team can state at least 3 evidence-backed findings and also identify at least 1 limitation or non-significant result.

Phase 8 — Adaptive Strategy Selection

Status: Conditional

Goal: Only if Phase 7 reveals meaningful instance-dependent behavior, build a transparent selector.

Tasks:

- Extract low-cost state features such as problem size, dynamicity, current route cost, number of newly released vertices and route-change magnitude.
- Define a small interpretable rule set or decision tree.
- Train/derive the selector only from a designated development set.
- Validate on held-out seeds/instances or held-out instance regimes.
- Count feature extraction and selection overhead.
- Compare adaptive selection against always using each fixed strategy.
- Measure selector regret relative to an oracle that knows the best strategy after the fact.
- If adaptation does not improve results, do not force it; retain the controlled-comparison study as the primary contribution.

Deliverables:

- Selector specification
- Selection logic
- Held-out validation results
- Fixed-vs-adaptive comparison
- Failure analysis

Phase gate: Adaptive method must beat or meaningfully trade off against fixed baselines on unseen conditions; otherwise it is removed from the core claim.

Phase 9 — Full UI, Visualization & Demonstration

Status: Next

Goal: Turn the validated research engine into a polished, understandable research demonstrator.

Tasks:

- Build a modern but minimal dashboard around the validated engine.
- Show the problem graph/map, current route, dynamic event arrival, route update and metrics.
- Use animations to explain algorithm behavior rather than decorative motion.
- Include a timeline/event panel showing what changed and why.
- Include side-by-side strategy comparison.
- Include live metric cards: route cost, update time, disruption and computational effort.
- Include a research mode with experiment results, plots and confidence intervals.
- Include a replay mode using recorded experiment seeds.
- Clearly distinguish simulation from real-world map data.
- Do not expose unfinished experimental controls in the final demo.

Deliverables:

- Final interactive UI
- Dynamic route animation

- Algorithm comparison view
- Experiment/result dashboard
- Demo scenario presets

Phase gate: A person who knows nothing about the project can understand the problem, observe a dynamic event, see the route change, and understand the measured trade-off without reading source code.

Phase 10 — Final Validation, Reproducibility & Submission Package

Status: Final

Goal: Freeze the research artifact and ensure that every displayed result can be regenerated.

Tasks:

- Freeze final code version.
- Re-run the final experiment matrix from clean environment/configuration.
- Regenerate all final plots directly from raw results.
- Verify every UI number against the analysis data.
- Run full test suite.
- Document hardware, Python version, dependencies, seeds and configurations.
- Prepare final presentation/demo flow.
- Prepare final project documentation and research-style report as part of the submission package, not as a separate development phase.
- Archive raw results and experiment manifests.

Deliverables:

- Final application
- Final result dataset
- Regeneration scripts
- Final presentation assets
- Research report/documentation
- Reproducibility package

Phase gate: A clean checkout can reproduce the core experiments and the final UI displays results traceable to saved experiment outputs.

6. Antigravity Operating Rules

- Read the project blueprint and existing repository before changing code.
- Work only on the current phase. Do not implement future phases unless explicitly requested.
- Do not silently change the mathematical model, objective, metrics, or experimental protocol.
- Prefer small, testable modules over a single large implementation.
- Every algorithm must have unit tests and deterministic behavior under a fixed seed where applicable.
- Every experiment must record configuration, seed, algorithm version/commit, runtime environment and raw metrics.
- Never hard-code research results into the UI. The UI must read generated result files.
- Never fabricate benchmark values, statistical significance, or performance claims.
- If an assumption is missing, stop and document the ambiguity instead of guessing.
- Keep a CHANGELOG.md entry for meaningful architecture or research-protocol changes.
- Before finishing a phase, run tests and produce the phase-specific acceptance evidence.
- Do not optimize code prematurely; first establish correctness, then measure, then optimize bottlenecks.

7. Repository Architecture

```
adaptive-route-optimization/
■■■ README.md
■■■ AGENTS.md
■■■ requirements.txt
■■■ pyproject.toml
■■■ docs/
■ ■■■ MASTER_BLUEPRINT.md
■ ■■■ RESEARCH_SPEC.md
■ ■■■ LITERATURE_MATRIX.md
■ ■■■ EXPERIMENT_PROTOCOL.md
■ ■■■ CHANGELOG.md
■■■ src/
■ ■■■ models/
■ ■■■ algorithms/
■ ■ ■■■ baseline/
■ ■ ■■■ local_search/
■ ■ ■■■ repair/
■ ■ ■■■ adaptive/
■ ■■■ simulation/
■ ■■■ evaluation/
■ ■■■ experiments/
■ ■■■ visualization/
■■■ tests/
■■■ configs/
■■■ data/
■ ■■■ generated/
■ ■■■ benchmarks/
■ ■■■ manifests/
■■■ results/
■ ■■■ raw/
■ ■■■ processed/
■ ■■■ figures/
■■■ notebooks/
■■■ app/
■■■ components/
■■■ pages/
■■■ assets/
```

8. Phase-Specific Antigravity Prompts

Paste only the prompt for the current phase. Do not give Antigravity all future phase prompts at once.

Phase 2

You are implementing Phase 2 of the Adaptive Route Optimization project. First inspect docs/MASTER_BLUEPRINT.md and the current repository. Do not implement algorithms or UI yet. Produce and commit a formal mathematical specification of the current dynamic routing model: graph, route, objective, dynamic event, state transition, feasibility, dynamicity, route disruption, computational budget placeholder, and evaluation metrics. Create pseudocode for the planned baseline, insertion repair, and 2-opt repair. Add small hand-checkable examples and tests for the definitions. If the repository contains ambiguity about single-route versus multi-route modeling, use the project default: single-route dynamic TSP for the first correct core and document the later multi-route extension as optional. Do not change the research question.

Phase 3

Implement Phase 3 only. Build a modular static routing engine from the approved Phase 2 specification. Implement graph/instance generation with deterministic seeds, route representation, route-cost calculation, feasibility checks, nearest-neighbor/greedy construction, and an exact solver for tiny instances only for correctness validation. Add unit tests and a simple route visualization. Do not implement dynamic events, adaptive selection, GA, SA, ALNS, or final UI. Finish by running the complete test suite and documenting the evidence.

Phase 4

Implement Phase 4 only. Add dynamic insertion repair and 2-opt local search to the validated static engine. Instrument candidate evaluations, accepted moves, iterations and runtime. Ensure both methods preserve route validity and are reproducible. Add adversarial edge-case tests. Do not implement the dynamic simulator or adaptive selector yet. Benchmark only on small controlled cases to verify correctness and behavior.

Phase 5

Implement Phase 5 only. Build an event-driven dynamic simulator around the validated algorithms. Generate a complete hidden instance, reveal an initial subset, generate reproducible dynamic arrivals, update the route after each event, and log every state

transition. Use identical instance/event sequences when comparing strategies. Add replay from seed and configuration. Add a basic animation/prototype only for debugging. Do not build the final UI or statistical experiment dashboard yet.

Phase 6

Implement Phase 6 only. Build the experiment runner and reproducibility infrastructure. Support configurable problem sizes, dynamicity levels, random seeds, algorithms and one explicit computational budget. Every run must write raw results plus a complete manifest. Add batch execution, failure recovery, result validation and deterministic replay. Do not interpret results or create an adaptive selector. Produce a pilot matrix first and report runtime before scaling the experiment.

Phase 7

Implement Phase 7 only. Build the research analysis pipeline from raw experiment outputs. Compute route quality, response/update time, computational effort, route disruption, feasibility and scalability. Generate convergence curves, quality-vs-time plots, dynamicity curves, scaling plots and paired comparisons. Include uncertainty summaries. Add ablations for insertion versus insertion+2-opt. Do not change algorithm parameters after seeing final results without recording the change as a new experiment.

Phase 8

Implement Phase 8 only and only if Phase 7 demonstrates meaningful instance-dependent strategy differences. Derive a small interpretable selector using inexpensive state features. Use a development/validation split that prevents leakage. Count selector overhead. Compare against each fixed strategy and an oracle analysis. If the selector does not improve the quality-computation trade-off on held-out conditions, document the negative result and do not force the adaptive component into the final claim.

Phase 9

Implement Phase 9 only after the research engine and result files are stable. Build a polished research dashboard with animated dynamic route updates, event timeline, algorithm comparison, metric cards, experiment plots, replay and explanation panels. The UI must consume saved outputs from the experiment engine and must never contain manually typed research results. Prioritize explanatory animation: show new event → affected route segment → candidate repair → chosen route → before/after metrics. Keep the interface application-neutral and avoid unnecessary map/API dependencies.

Phase 10

Implement Phase 10 only. Prepare the final reproducibility and submission package. Run the full clean experiment matrix from the final code version, regenerate all plots from raw data, validate every UI metric against the result files, run all tests, freeze dependencies and record environment information. Prepare the final documentation and presentation assets. Do not change the research protocol merely to improve final numbers; if a change is necessary, run a new documented experiment.

9. Final UI / Animation Specification

- Home / Problem View — Explain the dynamic routing problem visually. Show nodes, current route, pending nodes and the current optimization objective.
- Live Simulation — Animated event stream: new node appears → current route is highlighted → candidate insertion positions are shown → selected repair is animated → new route is displayed.
- Algorithm Lab — Run the same scenario with Greedy Repair, Repair + 2-opt, and the stronger method if included. Use synchronized replay so the difference is fair.
- Research Dashboard — Dynamicity vs cost, dynamicity vs update time, quality-vs-computation frontier, scaling curves, disruption metrics and statistical uncertainty.
- Adaptive Decision View — If Phase 8 succeeds, show the features used, selected strategy, alternatives and estimated trade-off. Never present the selector as magic AI.
- Replay — Load a saved seed/experiment ID and reproduce the exact event sequence and result.
- Methodology Drawer — Explain objective, algorithms, budget, metrics and benchmark source in plain language for evaluators.

10. Research Metrics

- Route cost / distance — Primary solution-quality metric.
- Update response time — Time required to produce a usable updated solution after a dynamic event.
- Computational effort — Function evaluations, iterations, candidate checks or the chosen standardized budget measure.

- Route disruption — How much of the existing route changed after an event.
- Feasibility — Whether the updated route satisfies all defined constraints.
- Scalability — How quality and computation change with instance size.
- Regret — For adaptive selection: difference between selected strategy and the best strategy for that instance/event.

11. Four-Person Team Division

- Member 1 — Core Algorithm & Model — Mathematical model, graph/route representation, baseline construction, 2-opt, correctness and complexity.
- Member 2 — Dynamic Simulation — Dynamic events, state transitions, route repair, replay, event logging and simulator tests.
- Member 3 — Experiments & Performance — Experiment runner, budgets, seeds, benchmarking, runtime instrumentation and reproducibility.
- Member 4 — Analysis & Product Layer — Statistical analysis, plots, adaptive-selector experiments, final UI/animation and documentation.
- Shared — Literature, problem explanation, algorithm understanding, final testing, presentation and research decisions.

12. 12-Week Execution Timeline

Time	Primary Work
Week 1	Phase 2: mathematical specification + hand examples
Week 2	Phase 3: static engine + correctness tests
Week 3	Phase 4: insertion + 2-opt + algorithm benchmarks
Week 4	Phase 5: dynamic simulator + replay
Week 5	Phase 6: pilot experiment framework
Week 6	Phase 6: full controlled experiment campaign
Week 7	Phase 7: analysis + ablation
Week 8	Phase 7: robustness / held-out validation
Week 9	Phase 8: adaptive selector if justified
Week 10	Phase 9: final UI + animation
Week 11	Phase 10: final rerun + reproducibility
Week 12	Phase 10: final demo, documentation and presentation preparation

13. Scope Boundaries

- Core: dynamic graph-based routing, reproducible event generation, route repair, local search, controlled re-optimization, experiments and analysis.
- Allowed later: stronger comparator, public routing benchmarks, multi-route extension, adaptive strategy selection.
- Not core: live GPS, map/traffic/weather APIs, reinforcement learning, deep learning, drone-specific logic, production dispatch, mobile app or cloud deployment.
- Delivery/courier is an example interpretation only; the technical engine remains application-neutral.

14. Fallback Strategy

- Stronger method too complex — Use a simpler budget-controlled re-optimization/local-search baseline. The controlled comparison remains valid.
- Adaptive selector not useful — Remove it from the core contribution. A rigorous controlled comparison is better than a forced weak 'AI' component.
- Runtime too high — Reduce pilot matrix, optimize measured bottlenecks, or use smaller sizes while preserving repeated controlled comparisons.

- Benchmark integration delayed — Complete synthetic controlled experiments first; add public benchmarks only if the core pipeline is already stable.
- UI consumes too much time — Freeze algorithm code and use the result-file-driven dashboard. Never sacrifice research validity for animations.
- Results are mixed — Do not hide them. Mixed results can directly answer the research question about when each strategy is useful.

15. Definition of Done

- Mathematical problem specified and reproducible.
- At least two route-update strategies implemented and tested.
- Stronger comparator used only if feasible and useful.
- Identical dynamic instances/events and controlled budgets used for comparisons.
- Quality, computation and disruption reported.
- Ablation and robustness/held-out validation completed where feasible.
- Adaptive selector included only if supported by evidence.
- Final UI reads actual experiment outputs; no hard-coded research numbers.
- Clean environment can reproduce final experiment outputs.
- All members can explain algorithms, complexity, metrics, results and limitations.

16. Final Project Positioning

The project is a research-oriented DAA study of dynamic combinatorial optimization. The central question is not simply which algorithm gives the shortest route, but which update strategy gives the best trade-off between route quality and computational response when the problem changes during execution, and whether observed behavior can support an interpretable adaptive choice.

17. Evidence Base / Source Files

- DAA_Adaptive_Route_Optimization_Master_Project_Blueprint.pdf — master scope and phase foundation.
- Papers on Algorithmic Routing Optimization (2019–2026).xlsx — curated routing literature corpus.
- Online_Parallel_optimization_Approach_to_Courier_Routing_Problems.pdf — online insertion, adaptive memory, SA and response-time comparison.
- Vista do A neighborhood search-based heuristic for the dynamic vehicle routing problem.pdf — dynamic VRP and 2-Opt* repair precedent.
- s41598-025-29861-y.pdf — traffic-aware dynamic routing, ALNS/Tabu/rollout, robustness and response-time evaluation.
- 2607.06066v1.pdf — recent dynamic dispatch comparison emphasizing quality versus computational cost.
- Pasted markdown(1).md and Pasted text.txt — literature synthesis, limitations and candidate research gaps.